SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING PRACTICAL ASSIGNMENT

1. Student Details

Student Name	Pratik Akrur Pandit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	26/10/2026

2. Practical Details

Practical No.	8
Practical Title	Exception Handling
Date Performed	21/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

Write a C++ program that implements exception handling operations using the try, throw and catch keywords. The program must throw and handle exceptions of different types (int, string, user-defined class and standard exception classes), and also demonstrate multiple catch blocks, a catch-all handler and re-throwing of an exception.

4. Objectives

1. To understand exceptions as run-time anomalies and how C++ handles them.
2. To use the try, throw and catch keywords to separate error-handling code from normal code.
3. To handle different types of exceptions, including user-defined exceptions, multiple catch blocks and the catch-all handler.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Write programs that detect errors using try blocks and throw exceptions when a problem occurs.
2. Catch and handle exceptions of different types using one or more catch blocks.
3. Create and use custom exception classes and re-throw exceptions to the calling function.

6. Theory / Concept

Exceptions are run-time anomalies or abnormal conditions that a program encounters during its execution, such as coding errors, wrong input or other unforeseeable things. When an error occurs, C++ normally stops and generates an error message; the technical term for this is that C++ *throws an exception*. Exceptions are of two types: **synchronous** and **asynchronous** (those beyond the program's control, such as disk failure). C++ exception handling is built on three keywords:

- **try**: represents a block of code that is tested for errors while it is being executed, and which can throw an exception.

- **throw**: used to throw an exception when a problem is detected, which lets us create a custom error. It is also used to list the exceptions that a function throws but does not handle itself.
- **catch**: represents a block of code that is executed when a particular exception is thrown in the try block. The catch statement takes a parameter whose type must match the type of the thrown exception.
  If no error occurs, the catch block is skipped.

The try and catch keywords come in pairs. Both basic types and objects can be thrown as exceptions. A function can throw many exceptions but may choose to handle only some of them; the rest are handled by its caller (or the caller's caller).

Advantages of exception handling over traditional error handling:

1. **Separation of error-handling code from normal code:** traditional if-else error checks get mixed with the normal flow, making code less readable and maintainable. With try-catch blocks, error handling is kept separate.
2. **Functions can handle any exceptions they choose:** uncaught exceptions are passed to the caller.
3. **Both basic types and objects can be thrown as exceptions.**

7. Algorithm / Pseudocode

```
START
Step 1: Define a custom exception class InvalidMarksException.
Step 2: Define functions checkAge(), divide(), validateMarks() and
        getElement() that throw an exception when a condition fails
        (int, string, object and out_of_range respectively).
Step 3: In main(), call each function inside a try block.
Step 4: Write a matching catch block for each exception type and display
        a suitable error message.
Step 5: Define innerFunction() that catches an exception, prints it and re-
        throws it using throw; ; catch it again in main().
Step 6: Throw a double value and handle it with multiple catch blocks,
where the catch(...) block handles any type not matched earlier. Step 7:
Display the output and end the program.
STOP
```

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++ (C++11 standard)
IDE / Platform	Terminal / text editor on Linux
Compiler	g++ (GCC)
Libraries / Packages	<iostream> (input/output), <string> (string type), <stdexcept> (standard exception classes)

09. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**
Branch Used	main
Commit ID / Latest Commit	7f8fc4031843bc9245fef14b740b25841f3d265b
Date of Last Commit	26/10/26

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1 checkAge()	age = 20	Access granted. Age is 20	Access granted. Age is 20	Pass
TC2 checkAge()	age = 15	Access denied. Age is 15 (must be 18 or above)	Access denied. Age is 15 (must be 18 or above)	Pass
TC3 divide()	10, 2	10 / 2 = 5	10 / 2 = 5	Pass
TC4 divide()	10, 0	Exception caught: Division by zero is not allowed	Exception caught: Division by zero is not allowed	Pass
TC5 validateMarks()	marks = 85	Marks 85 are valid	Marks 85 are valid	Pass
TC6 validateMarks()	marks = 120	Invalid marks entered: 120 (must be between 0 and 100)	Invalid marks entered: 120 (must be between 0 and 100)	Pass
TC7 getElement()	index = 1	arr[1] = 20	arr[1] = 20	Pass
TC8 getElement()	index = 5	Exception caught: Index 5 is out of range	Exception caught: Index 5 is out of range	Pass
TC9 re-throw	runtime_error	Inner catch: ... / Outer catch: Error occurred in inner function	Inner catch: ... / Outer catch: Error occurred in inner function	Pass
TC10 catch-all	throw 3.14	Unknown (default) exception caught	Unknown (default) exception caught	Pass

11. Results / Observations

- When an exception was thrown inside a try block, the remaining statements of that block were skipped and control moved to the matching catch block (for example, the line after divide(10, 0) was never executed).
- When no error occurred (age 20, marks 85), the catch blocks were skipped and the program ran normally.
- Exceptions of type int, string, a user-defined class and the standard out_of_range class were all caught successfully by catch blocks with matching parameter types.
- A re-thrown exception was caught first by the inner handler and then again by the outer handler in main().
- The catch(...) handler caught the double value, which did not match any of the earlier catch blocks, so the program ended normally instead of crashing.

Screenshots / Graphs / Output:

Pratical_assignment_8.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

// Custom exception class
class InvalidMarksException {
    int marks;

public:
    InvalidMarksException(int m) : marks(m) {}

    void show() const {
        cout << "Invalid marks entered: " << marks
             << " (must be between 0 and 100)" << endl;
    }
};

// Functions that throw exceptions
void checkAge(int age) {
    if (age < 18)
        throw age;

    cout << "Access granted. Age is " << age << endl;
}

int divide(int a, int b) {
    if (b == 0)
        throw string("Division by zero is not allowed");

    return a / b;
}

void validateMarks(int marks) {
    if (marks < 0 || marks > 100)
        throw InvalidMarksException(marks);

    cout << "Marks " << marks << " are valid" << endl;
}

int getElement(const int arr[], int size, int index) {
    if (index < 0 || index >= size)
        throw out_of_range(
            "Index " + to_string(index) + " is out of range"
        );

    return arr[index];
}
```

```
48  }
49
50  void innerFunction() {
51      try {
52          throw runtime_error("Error occurred in inner function");
53      }
54      catch (const runtime_error &e) {
55          cout << "Inner catch: " << e.what() << endl;
56          throw;
57      }
58  }
59
60  int main() {
61      cout << "===== 1. try, throw and catch (int) =====" << endl;
62
63      try {
64          checkAge(20);
65      }
66      catch (int myNum) {
67          cout << "Access denied. Age is " << myNum
68               << " (must be 18 or above)" << endl;
69      }
70
71      try {
72          checkAge(15);
73      }
74      catch (int myNum) {
75          cout << "Access denied. Age is " << myNum
76               << " (must be 18 or above)" << endl;
77      }
78
79      cout << endl << "===== 2. Throwing a string =====" << endl;
80
81      try {
82          int r1 = divide(10, 2);
83          cout << "10 / 2 = " << r1 << endl;
84
85          int r2 = divide(10, 0);
86          cout << "10 / 0 = " << r2 << endl;
87
88          cout << "This line is never executed" << endl;
89      }
90      catch (const string &msg) {
91          cout << "Exception caught: " << msg << endl;
92      }
93
```

```
    cout << endl << "===== 3. Custom exception class =====" << endl;

    try {
        validateMarks(85);
        validateMarks(120);
    }
    catch (const InvalidMarksException &e) {
        e.show();
    }

    cout << endl
         << "===== 4. Standard exception (out_of_range) ====="
         << endl;

    int arr[3] = {10, 20, 30};

    try {
        int v1 = getElement(arr, 3, 1);
        cout << "arr[1] = " << v1 << endl;

        int v2 = getElement(arr, 3, 5);
        cout << "arr[5] = " << v2 << endl;
    }
    catch (const exception &e) {
        cout << "Exception caught: " << e.what() << endl;
    }

    cout << endl << "===== 5. Re-throwing an exception =====" << endl;

    try {
        innerFunction();
    }
    catch (const exception &e) {
        cout << "Outer catch: " << e.what() << endl;
    }

    cout << endl
         << "===== 6. Multiple catch blocks and catch-all ====="
         << endl;

    try {
        throw 3.14;
    }
    catch (int) {
        cout << "Integer exception caught" << endl;
    }
    catch (string) {
    }
    catch (int) {
        cout << "Integer exception caught" << endl;
    }
    catch (string) {
        cout << "String exception caught" << endl;
    }
    catch (...) {
        cout << "Unknown (default) exception caught" << endl;
    }

    cout << endl << "Program ended normally." << endl;

    return 0;
}
```

- **Output:-**

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== 1. try, throw and catch (int) =====
Access granted. Age is 20
Access denied. Age is 15 (must be 18 or above)

===== 2. Throwing a string =====
10 / 2 = 5
Exception caught: Division by zero is not allowed

===== 3. Custom exception class =====
Marks 85 are valid
Invalid marks entered: 120 (must be between 0 and 100)

===== 4. Standard exception (out_of_range) =====
arr[1] = 20
Exception caught: Index 5 is out of range

===== 5. Re-throwing an exception =====
Inner catch: Error occurred in inner function
Outer catch: Error occurred in inner function

===== 6. Multiple catch blocks and catch-all =====
Unknown (default) exception caught

Program ended normally.
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

In this practical, a C++ program was implemented to demonstrate exception handling using the try, throw and catch keywords. Exceptions of basic types, strings, user-defined classes and standard exception classes were thrown and handled, and multiple catch blocks, the catch-all handler and re-throwing were also used. I learned that exception handling keeps error-handling code separate from normal code, which makes programs more readable, maintainable and robust.

13. Assignment Questions

Q1. Explain the try, throw and catch keywords in C++.

Answer: The **try** block contains the code that is tested for errors while it is being executed. The **throw** keyword is used to throw an exception when a problem is detected; the thrown value can be a basic type or an object. The **catch** block defines the code to be executed when an exception of the matching type is thrown in the try block. The catch statement takes a parameter whose type must match the type of the exception thrown. try and catch always come in pairs, and if no exception occurs the catch block is skipped.

Q2. What are the advantages of exception handling over traditional error handling?

Traditional error handling	Exception handling
Error checks (if-else) get mixed up with normal code	Error-handling code is separate from the normal flow
Code becomes less readable and maintainable	Code is cleaner, readable and easier to maintain

Traditional error handling	Exception handling

Every function has to check and handle errors itself	A function can handle only the exceptions it chooses; the rest are passed to the caller
Usually limited to error codes	Both basic types and objects can be thrown

Q3. What happens if an exception is thrown but not caught? What is the use of catch(...)?

Answer: If an exception is not caught by any handler in the function or its callers, the program terminates abnormally (C++ calls terminate()). The catch(...) block, called the catch-all handler, can catch an exception of any type. It is written after the specific catch blocks and handles any exception that did not match them, so the program does not crash.

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Pratik Akrur Pandit	202501110116	